

main_ssi

March 26, 2026

```
[2]: import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torch.optim import Adam

from sklearn.model_selection import train_test_split

from TorchSpatial.trainer import train, train_sri_debias
from TorchSpatial.test import test
from TorchSpatial.modules.encoder_selector import get_loc_encoder
from TorchSpatial.modules.models import ThreeLayerMLP
import TorchSpatial.utils.datasets as data_import
import TorchSpatial.utils.eval_helper as eval_helper

from gbsloss import SSIPartitioner, BinaryPerformanceTransformer, SSILoss, \
    ↪SRIPartitioner, SoftHistogramPerformanceTransformer, SRILoss

from pathlib import Path
import numpy as np
import pandas as pd

import torch
import numpy as np

import json

import warnings
```

```
[3]: all_data = data_import.load_dataset(params = {"dataset": "birdsnap", \
    ↪"meta_type": "ebird_meta", "regress_dataset": []},
    eval_split = "test",
    train_remove_invalid = True,
    eval_remove_invalid = False,
    load_cnn_predictions=True,
    load_cnn_features=False,
    load_cnn_features_train=False)
```

Loading birdsnap_with_loc_2019.json - train

```

using meta data: ebird_meta
46386 total entries
43426 entries with images
42490 entries with meta data
Loading birdsnap_with_loc_2019.json - test
using meta data: ebird_meta
2443 total entries
2262 entries with images
2217 entries with meta data
keeping entries even without metadata

```

```
[4]: all_data.keys()
```

```
[4]: dict_keys(['train_locs', 'val_locs', 'train_preds', 'train_classes',
'train_users', 'train_dates', 'train_inds', 'train_imgs', 'val_classes',
'val_users', 'val_dates', 'val_inds', 'val_imgs', 'class_of_interest',
'classes', 'num_classes', 'val_preds', 'val_split'])
```

```
[5]: all_data["train_preds"].shape # This is the cnn image prediction for training
```

```
[5]: (43426, 500)
```

```
[6]: all_data["val_preds"].shape # This is the cnn image prediction for validation
```

```
[6]: (2262, 500)
```

```
[11]: all_data["val_preds"][170]
```

```
[11]: array([0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 1.7273960e-06, 4.1142102e-05, 2.7263520e-06,
5.0794383e-06, 1.5173841e-04, 0.0000000e+00, 0.0000000e+00,
1.8991213e-04, 7.1914430e-05, 1.1283653e-06, 4.3496152e-06,
1.9363214e-04, 2.2858849e-02, 2.4051279e-04, 1.4847105e-03,
9.7452521e-01, 3.3960200e-06, 2.7185619e-05, 2.0818118e-06,
1.0291435e-06, 2.9843966e-06, 2.2626321e-06, 1.0603761e-06,
1.3172955e-06, 1.0495486e-06, 0.0000000e+00, 5.7713019e-06,
4.3320383e-06, 3.4168211e-06, 1.6155674e-06, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 5.0909803e-06, 1.4214729e-06, 1.3419848e-06,
2.4669807e-05, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 2.2550191e-06, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
```

[illegible]

[illegible]

```

0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00,
0.0000000e+00, 0.0000000e+00, 0.0000000e+00, 0.0000000e+00],
dtype=float32)

```

```
[10]: all_data["val_preds"][170].argmax()
```

```
[10]: np.int64(40)
```

```
[13]: all_data["val_preds"][170][all_data["val_preds"][170].argmax()] # max proba
```

```
[13]: np.float32(0.9745252)
```

```
[15]: all_data["val_preds"][3].max() # max proba
```

```
[15]: np.float32(0.6652693)
```

```
[16]: all_data["val_preds"][170].argmax() == all_data["val_classes"][170] # compare
      ↪ cnn preds to correct class, find accuracy
```

```
[16]: np.True_
```

```
[75]: import random
import numpy as np
truths = []

# Accuracy of 5000 randomly sampled pairs with replacement
for _ in range(5000):
    i = random.randint(0, len(all_data["val_preds"]) - 1)
    truths.append(all_data["val_preds"][i].argmax() ==
                  ↪ all_data["val_classes"][i])
truths = np.array(truths)
print(f"accuracy: {len(truths[truths == True]) / len(truths)}")

```

```
accuracy: 0.6998
```

```
[ ]: pred = class_probabilities.argmax(dim=1)
hit_at_1 = (pred == y_idx)
```

I reported seeing some confusion among the audience, who demanded a change of names so they may better recognize the variables. I completel agreed with them, so I likewise demanded the change,

and subsequently altered the old context to fit the new faces. They looked better immediately, and I was impressed of myself for having pulled this off all live on stage.

```
[97]: class_probab = torch.Tensor([all_data["val_preds"][5],  
    ↪all_data["val_preds"][6], all_data["val_preds"][7]])  
print(class_probab.shape)  
pred = class_probab.argmax(dim=1)  
y_idx = torch.Tensor([all_data["val_classes"][5], all_data["val_classes"][6],  
    ↪all_data["val_classes"][7]])  
hit_at_1 = (pred == y_idx)  
print(hit_at_1)
```

```
torch.Size([3, 500])  
tensor([False, False, False])
```

```
[182]: top1accuracies = []  
  
for _ in range(100):  
    class_probab = []  
    y_idx = []  
  
    for _ in range(200):  
        i = random.randint(0, len(all_data["val_preds"]) - 1)  
        class_probab.append(all_data["val_preds"][i])  
        y_idx.append(all_data["val_classes"][i])  
  
    pred = torch.tensor(class_probab).argmax(dim=1)  
    y_idx_tensor = torch.tensor(y_idx)  
    hit_at_1 = (pred == y_idx_tensor)  
  
    # print(hit_at_1[:6])  
    top1accuracy = hit_at_1.float().mean().item()  
    top1accuracies.append(top1accuracy)  
  
    # print(f"Top 1 Accuracy: hit_at_1 {top1accuracy:.4f}")  
    # print((hit_at_1 == True).sum().item() + (hit_at_1 == False).sum().item(),  
    ↪== len(hit_at_1))  
  
print(top1accuracies[:6])  
print(np.mean(top1accuracies))
```

```
[0.7300000190734863, 0.7049999833106995, 0.699999988079071, 0.7200000286102295,  
0.6899999976158142, 0.6800000071525574]  
0.6961000025272369
```

Seeing my code being glowed up by AI, I feel both relieved and confused. It was not in my own style. It is different. I must read it again.

```
[ ]: top1accuracies = []
```

In front of the vast and empty space of the background of a comfortably colored darkness, I carefully, meticulously, precisely arranged a decent room amidst the orderly chaos in the random access memory in preparation for the soon-expected growth in registered membership count belonging to the name that I have just previously decided and set. I prepared it in response to the prophecy on the birth of many who will be of immense value to the mission I have accepted and undertook, whom I must seek and gather immediately after their arrival, lest I return an iteration later and fail to find them, only to realize their unfortunate but predetermined erasure by next-generation inhabitants arriving carrying the exact same names.

```
[ ]: for _ in range(100):
```

I dispatched a trivial messenger, a variable whose name might as well be nonexistent, so the counter for the loop may work. Like all good variables, it would firmly hold onto its assigned value upon each announcement, to carry its important flag of one specific integer to synchronize the knowledge of the state of iterations for all who would demand to receive an update on the most recent progress.

(for i would be adding: “so it might hop through the various reserved positions hidden in the maze-like steps installed into the subsequent lines of the looping program”)

```
[ ]: i = random.randint(0, len(all_data["val_preds"]) - 1)
```

In my right hand I hold a form that I wish I can already send out. But until things spin into motion, until I am granted live permission to measure the length of this specific division of the data, I will not have my upper bound, which is a necessary piece of information that will be requested from my submission by the Office of the Library of Artificial Randomness once they begin to process my “Random Integer Request”. Until then, until the office would return its integral response, the identity of i would remain a bounded mystery, and I can only wait and press down my burning desire to unveil it.

```
[ ]: class_probab.append(all_data["val_preds"][i])
     y_idx.append(all_data["val_classes"][i])
```

I would command, “Come out, you the i-th member of the list belonging to the “val_preds” division of the all_data group!” and it would come out from its place. It is a list of mutually exclusive probabilities, as expected. I would append it to the end of the class_probab group, the collection I built for all whom I will sample from the same division of the same group. I would do similarly for the class labels, calling “you i-th member of the list belonging to the “val_classes” division of the all_data group” and append it to another group – y_idx. Each would abide by my order and stay in its place. In fact, I would be quite terrified if any one of them moved on its own, for that would imply that another secret commander – or disrupter – or confounding factor, was also present.

```
[ ]: preds = torch.tensor(class_probab).argmax(dim=1)
```

It is unfortunate that I must proceed with an immigration request, but a Python list is truly too primitive to support even an operation such as an argmax with a dimensional specification. Fortunately, in the past, as long as I had correctly sent over what it could process, torch’s tensor conversion unit had always received my input, performed the argmax search, and returned a single array of single integers, each of which indicated the position of the largest probability in each list of probabilities which lie along dimension one, all flawlessly without fail. I would gladly receive its processed tensor, and nickname it “preds”.

```
[ ]: y_idx_tensor = torch.tensor(y_idx)
```

y_idx passed through the gate of conversion, and subsequently received a suffix indicating its new data structure, a suffix that hints toward a different version of itself that exists somewhere else.

```
[ ]: hit_at_1 = (pred == y_idx_tensor)
```

The two tensors named pred and y_idx_tensor were aligned. Sharing the same shape allowed them to bijectively correspond their entries and thus compare element-wise. The boolean rules governing the comparison are simple:

Let a match be True, a mismatch be False;

Collect them all, and keep the shape;

At each entry, its status lays;

Name it hit_at_1, so it stays.

```
[ ]: toplaccuracy = hit_at_1.float().mean().item()
```

Before it is just a number, it was a tensor holding the number; before it held that single number, it held many numbers; before it held the many numbers, it held them as boolean markers. Through a sequence of transformational portals it became what it is, a Python float rightfully named toplaccuracy, representing the percentage of correctness that is the aggregate of every individual True and False divided by the size of the selected batch, according to one of the strictest, merciless criterion of judgement that can be reasonably implemented, analogous to a multiple choice response that is restricted to a single option only: hit_at_1.

```
[ ]: toplaccuracies.append(toplaccuracy)
```

The list mechanically received another number, oblivious to the programmer's intent in heart.

```
[ ]: print(toplaccuracies[:6])
```

In order to prevent revealing a whole screen of numbers, I restricted to display only the first six elements of the list. Not everything stored must be seen.

```
[ ]: print(np.mean(toplaccuracies))
```

I averaged across the floats in the list called toplaccuracies, invisibly making a numpy copy in the process.

```
[90]: class_probas_based_on_image = torch.from_numpy(all_data["val_preds"][5])  
      class_probas_based_on_image[:6]
```

```
[90]: tensor([0.0010, 0.0010, 0.0058, 0.0132, 0.0043, 0.0030])
```

```
[89]: class_probas_based_on_loc = loc_embedding = torch.  
      ↪ones_like(class_probas_based_on_image).float()  
      class_probas_based_on_loc[:6]
```



```
[89]: tensor([1., 1., 1., 1., 1., 1.])
```

```
[91]: (class_probas_based_on_image * class_probas_based_on_loc)[:6]
```

```
[91]: tensor([0.0010, 0.0010, 0.0058, 0.0132, 0.0043, 0.0030])
```

```
[93]: all(class_probas_based_on_image * class_probas_based_on_loc ==  
        ↪class_probas_based_on_image) # simulating no_prior. Should be true before  
        ↪and after, because loc_probas are all 1.
```

```
[93]: True
```

```
[ ]:
```